NEURAL NETWORK TRAINING WALKTHROUGH — 45-MINUTE PRESENTATION SCRIPT
==========

INTRODUCTION (2 minutes)

Hello everyone! Today we'll walk step-by-step through a complete neural■network training pipeline using PyTorch.
We will explore loading data, preprocessing, designing a neural network, tuning hyperparameters, training, evaluating,
and interpreting results. This script closely follows the code you see in the notebook.

Throughout this session, I will also highlight how changing each hyperparameter impacts model learning behavior.

---

0. CONFIG & IMPORTS (3 minutes)

We begin the notebook with several import statements:

• pandas, numpy → for data loading and transformations
• matplotlib, seaborn → for plotting distributions and correlations
• torch, torch.nn, torch.optim → for building and training neural networks
• DataLoader, TensorDataset → for batching and loading data efficiently
• StandardScaler → for feature scaling
• sklearn.metrics → for evaluating classification model performance

We also set SEED = 42 for reproducibility.
Reproducibility is very important because neural networks involve randomness (initial weights, batch sampling).

Hyperparameters defined here:

• ACTIVATION – activation function inside network layers
• HIDDEN_SIZES – number of hidden layers + their sizes
• LEARNING_RATE – how quickly weights update
• OPTIMIZER_NAME – SGD or Adam
• NUM_EPOCHS – number of full passes through training data
• USE_BATCH_NORM – whether to use batch normalization
• DROPOUT_P – dropout probability
• VAL_RATIO & TEST_RATIO – dataset splitting fractions

How changing these hyperparameters affects training:
• Increasing HIDDEN_SIZES increases model capacity → can learn more but may overfit.
• Learning rate too high → unstable training; too low → slow learning.
• Adam usually converges faster than SGD.
• More epochs → more training but may overfit after a point.
• Batch Norm stabilizes training, especially with deep networks.
• Dropout reduces overfitting by randomly turning off neurons.

---

1. LOAD DATA (3 minutes)

We load **Insurance.csv** and print:

• head() – first few rows
• info() – data types
• missing value counts
• summary statistics

This helps us understand the structure:
age, bmi, children, charges are numeric features
sex, smoker, region, insuranceclaim are categorical

insuranceclaim is our target variable.

---

2. EXPLORATORY DATA ANALYSIS (5 minutes)

Here we show histograms of numeric features.
We visualize distributions to identify skew, range, or outliers.

We also plot bar charts for categorical features.
This helps check class imbalance – very important for classification.

Next, we generate a correlation heatmap.
We might see:

• charges strongly correlated with smoker status
• age correlated with bmi or charges

Then a boxplot of charges vs smoker shows that smokers pay far higher charges.
This also shows why that feature is important.

---

3. PREPROCESSING (5 minutes)

The preprocess() function performs three major tasks:

1. Splitting features and target
2. One■hot encoding categorical columns
3. Scaling numeric features using StandardScaler

Why scaling matters:
Because neural networks converge faster when inputs have similar magnitudes.

One■hot encoding converts sex, region, smoker into numeric columns.

Potential effect of preprocessing changes:
• If you skip one■hot encoding → model cannot understand categorical text and will fail.
• If you skip scaling → gradients may explode or vanish.

We convert everything into tensors for PyTorch.

---

4. DATASET SPLIT & LOADERS (3 minutes)

We split dataset using random_split() into:

• Training
• Validation (used to tune hyperparameters)
• Test set (used only for final evaluation)

DataLoaders handle batching.
BATCH_SIZE controls how many samples each step uses.

Changing batch size:
• Small batch → more noisy gradients but may generalize better
• Large batch → faster but may get stuck in sharp minima

---

5. MODEL BUILDING (6 minutes)

We define a class NeuralNet which builds a feed■forward network dynamically based on hyperparameters.

Layers added:
• Linear → BatchNorm (optional) → Activation → Dropout (optional)

Activation options:
• ReLU – fast, widely used
• LeakyReLU – helps with dying ReLU problem
• Tanh – good for small networks
• Sigmoid – rarely used in hidden layers

How activation affects training:
• ReLU → sparse gradients, fast
• Tanh/Sigmoid → saturation risk → slow training
• LeakyReLU → keeps gradient nonzero even for negative values

Dropout:
Reduces overfitting by randomly disabling fractions of neurons each iteration.

Batch Norm:
Normalizes intermediate activations → faster, more stable training.

Output layer: Linear (logits) because CrossEntropyLoss applies softmax internally.

---

6. LOSS FUNCTION & OPTIMIZER (4 minutes)

Loss: CrossEntropyLoss
This is standard for multi■class (and binary) classification with logits.

Optimizer:
• SGD – simple, may need tuning
• Adam – adaptive learning rate, usually converges faster

Learning rate impact:
• Too high → divergence
• Too low → slow convergence

---

7. TRAINING & VALIDATION LOOPS (7 minutes)

During training mode:

1. Forward pass
2. Compute loss
3. Backward pass (loss.backward())
4. Weight update (optimizer.step())

Validation:
• No gradient updates
• We track loss + accuracy

We store history for plotting.

Why validation matters:
To detect **overfitting** (train loss ↓ but val loss ↑).

---

8. PLOTTING TRAINING CURVES (2 minutes)

We visualize:

• Train vs validation loss
• Validation accuracy over epochs

Interpretation:
• If train loss ↓ but val loss ↑ → model is overfitting
• If both flat → learning rate too low or model too small
• If oscillating → learning rate too high

---

9. EVALUATION ON TEST SET (4 minutes)

Compute:

• Accuracy
• Classification report (precision, recall, F1-score)
• Confusion matrix heatmap

This tells how well model generalizes.

---

10. DEMO PREDICTION (2 minutes)

We take a single sample from the test set and predict the insurance claim.

Shows how to use the model for inference.

CONCLUSION (2 minutes)

We covered:

• Loading + understanding data
• EDA
• Preprocessing
• Neural network architecture
• Training loops
• Hyperparameter tuning
• Evaluation
• Predictions

This whole pipeline is what modern ML workflows look like in real■world companies.